HOWARD COMMUNITY COLLEGE

Arduino Digital Piano

Technical Design and Implementation Report

ENES100 Introduction to Engineering Design

Student: AFFAN FAHIM KHAN

Date of Submission: October 8, 2026

Instructor: Dr. ZHEN LIU

Eight-key C-major piano (C4–C5), two programmed melodies, and animated 16×2 LCD

Contents

1 Introduction and Project Objective **2**

2 Electrical Engineering and Music Theory Background **2**

2.1 Digital inputs and button logic 2

2.2 Frequency and musical pitch 2

3 Materials, Components, and Pin Assignments **3**

3.1 Wiring summary 4

4 Design Process and Development **4**

4.1 Simulation and real-circuit documentation 5

4.2 Challenges Encountered 6

5 Circuit Schematic and Electrical Explanation **6**

6 Program Logic, Pseudocode, and Explanation **7**

6.1 High-level pseudocode 7

6.2 Main functions 7

6.3 Readable code and software design decisions 8

7 Results, Verification, and Rubric Alignment **8**

8 Reflection on ChatGPT-Assisted Development **8**

9 Conclusion **9**

A Complete Arduino Sketch **9**

B Code Walkthrough by Section **17**

C TinkerCad Project Code and Physical Lab Demonstration **17**

1 Introduction and Project Objective

The goal of this mini-project is to design and program an Arduino-based digital piano that allows a user to play the C-major scale on a breadboard. The required range begins at middle C (C4) and ends at the next C (C5). Eight physical note buttons provide this full octave. In addition to the basic scale, the design includes two buttons for automatic melodies, a 16×2 character LCD, and three custom-character frames used to animate a dancing figure while a song plays.

The project combines basic circuit design, microcontroller input/output, programming logic, and music theory. Each manual pushbutton represents a note. When pressed, the Arduino reads its digital input and drives a passive piezo sounder at the corresponding frequency. The LCD identifies the active note and its frequency; when the button is released, the screen returns to the idle message. Separate song buttons play an ABC Alphabet Song melody and "Mary Had a Little Lamb."

Design objectives: (1) implement all eight notes C4 through C5; (2) produce audible, distinct pitches; (3) display the selected manual note; (4) offer two preprogrammed songs; and (5) improve the visual presentation with an LCD animation.

2 Electrical Engineering and Music Theory Background

2.1 Digital inputs and button logic

The eight manual note switches use the Arduino's `INPUT` mode with external 10-kΩ pull-down resistors. A released switch is read as `LOW`; pressing the switch connects the input to 5 V, producing `HIGH`. The pull-down resistor prevents an unpressed input from floating and causing unintended notes. The two song switches use `INPUT_PULLUP`: they normally read `HIGH` and read `LOW` when pressed and connected to ground. These are different, intentional input conventions.

2.2 Frequency and musical pitch

A musical note corresponds to a periodic vibration. Frequency, measured in hertz (Hz), describes how many oscillations occur each second. The Arduino `tone()` function generates a square-wave signal at the selected frequency on the piezo output pin. In equal temperament, adjacent semitones are related by

$$f(n) = 440 \cdot 2^{(n-69)/12}, \tag{1}$$

where n is the MIDI note number and A4 (MIDI 69) is 440 Hz. For example, C5 is one octave above C4, so its frequency is approximately twice that of C4: $523.25 \approx 2(261.63)$ Hz. The code uses rounded integer frequencies appropriate for `tone()`.

Table 1: Required C-major scale and corresponding Arduino connections

Key	Note	Solfège	Frequency (Hz)	Pin
1	C4	Do	262	D9
2	D4	Re	294	D8
3	E4	Mi	330	D7
4	F4	Fa	349	D6
5	G4	Sol	392	D5
6	A4	La	440	D4
7	B4	Ti	494	D3
8	C5	Do	523	D12

The C-major scale uses the white-key notes C, D, E, F, G, A, B, and C. The eighth note is necessary to complete the octave specified by the assignment.

3 Materials, Components, and Pin Assignments

Component	Quantity	Purpose
Arduino Uno-compatible board	1	Reads buttons, produces sound, controls LCD
Momentary pushbuttons	10	Eight manual notes and two song selections
470Ω resistors	8	External pull-downs for manual note buttons
Passive piezo buzzer/sounder	1	Converts generated electrical waveform into audible notes
16×2 HD44780-compatible LCD	1	Displays idle message, notes, song titles, and animation
10-kΩ potentiometer	1	LCD contrast adjustment via VO
Breadboard and jumper wires	As needed	Physical assembly and interconnection
USB cable / 5-V supply	1	Powers and programs the board
LCD backlight resistor	If needed	Limits backlight current when not built into LCD module

3.1 Wiring summary

Signal	Arduino connection
Piezo positive	D2 (piezo negative to GND)
ABC Song button	D10 to GND; internal pull-up
Mary song button	D11 to GND; internal pull-up
Manual notes	D9,D8,D7,D6,D5,D4,D3,D12; external pull-downs
LCD RS, E	A0, A1
LCD DB4, DB5, DB6, DB7	A2, A3, A4, A5
LCD RW	GND
LCD VCC / GND	5 V / GND
LCD VO	Potentiometer wiper

4 Design Process and Development

The design begins with the assignment's functional requirement: a playable C-major octave. Each manual button is mapped to a distinct Arduino digital pin and to one note-frequency constant. The output stage uses a passive piezo and the Arduino `tone()` function. To improve usability, an LCD was added to show the currently pressed note, solfège label, and frequency. The design was then extended with two automatic song buttons and a small dancing-character animation.

An important debugging issue involved the C5 input on D12. When the input logic and physical wiring were inconsistent, C5 could appear to be pressed continuously and stop when the button was pushed. The final sketch treats C5 like the other manual keys: `INPUT` mode, an external pull-down resistor, and `HIGH` when pressed. This illustrates why the software input configuration must match the actual circuit.

4.1 Simulation and real-circuit documentation

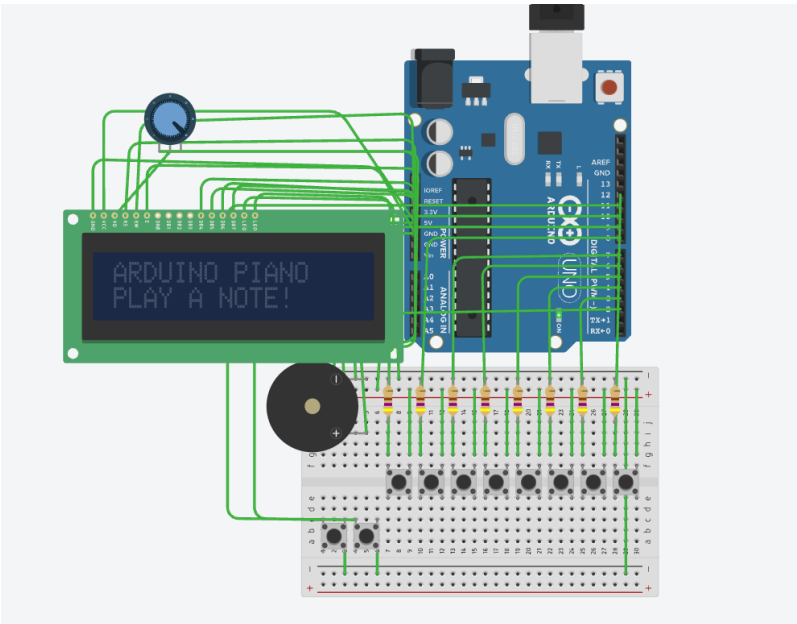


Figure 1: Tinkercad simulation screenshot showing the assembled piano circuit. Replace this placeholder with an actual screenshot.

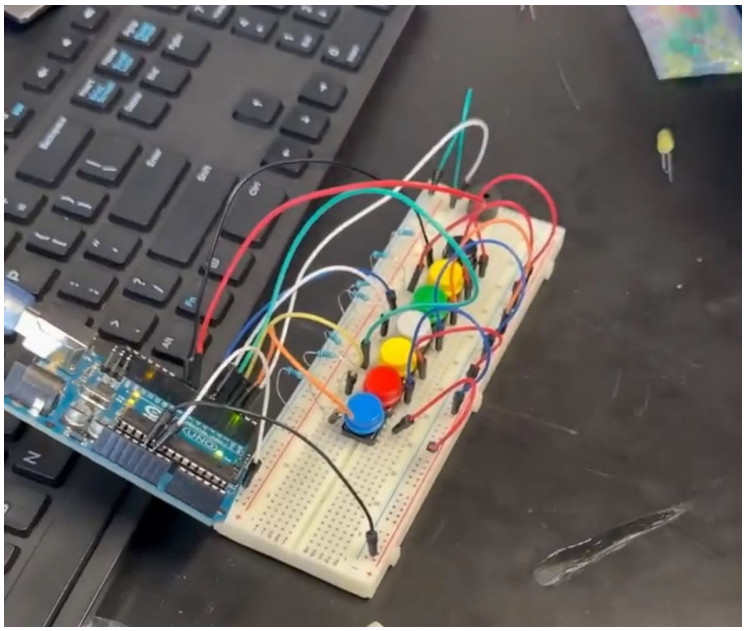


Figure 2: Photograph of the physical breadboard assembly

4.2 Challenges Encountered

Challenges During Circuit Development

During the development and testing of the Arduino Piano, several challenges were encountered involving button input detection and hardware functionality. One of the main issues was related to the C5 button connected to Arduino digital pin D12. Initially, the button behaved in the opposite way from what was expected: the buzzer produced the C5 note when the button was not pressed, and the sound stopped when the button was pressed. This occurred because the input configuration did not match the pull-down resistor arrangement used for the other seven piano keys. To resolve this issue, the pin configuration was changed to `INPUT`, and the program was modified to recognize `HIGH` as a pressed button. After making these corrections, the C5 button operated consistently with the other piano keys.

Another challenge occurred during the physical implementation of the circuit in the laboratory. The LCD screen provided with the physical Arduino kit was not functioning properly and experienced hardware-related issues that prevented it from displaying information reliably. Although the Arduino program included LCD functionality to display musical notes, frequencies, and a dancing character during automatic song playback, this feature could not be demonstrated successfully using the physical LCD. Therefore, the LCD screen was not included in the final physical laboratory demonstration. However, the LCD functionality remains implemented in the Arduino code as part of the overall circuit design.

5 Circuit Schematic and Electrical Explanation

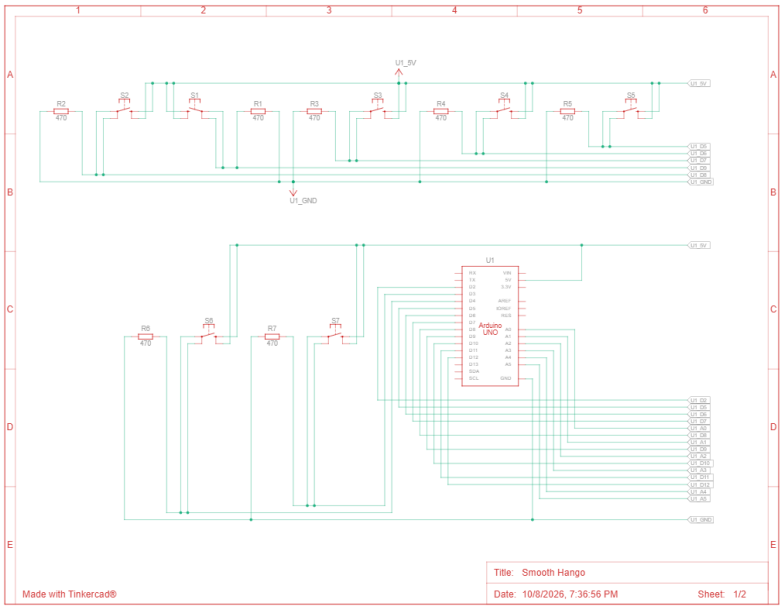


Figure 3: TinkerCad electrical schematic of the final Arduino piano circuit.

The schematic should show each manual input pulled down to ground through approximately 470Ω and connected to 5 V through its pushbutton. D12 must follow this same arrangement in the final version. The automatic-song inputs are different: their switches connect D10 and D11 to ground and use the microcontroller's internal pull-up resistors. The piezo is driven from D2, and the LCD uses a four-bit parallel data interface to conserve Arduino pins. The LCD contrast potentiometer adjusts the VO voltage; RW is tied to ground because the sketch only writes to the display. All parts share a common ground.

6 Program Logic, Pseudocode, and Explanation

6.1 High-level pseudocode

1. Configure eight manual buttons as digital inputs with external pull-downs; configure two song buttons as internal pull-up inputs.
2. Configure the piezo output, initialize the 16×2 LCD, and register three custom dancing-character frames.
3. Display "ARDUINO PIANO / PLAY A NOTE!".
4. Repeatedly read the two song buttons. If the ABC button has just been pressed, play the ABC melody; otherwise, if the Mary button has just been pressed, play that melody.
5. During an automatic song, step through the melody and duration arrays, sound each pitch, and update the LCD character frame.
6. When no song button is active, check the eight manual keys in order. For the first detected pressed key, select its frequency, name, and solfège label.
7. Sound the selected note and update the LCD only when the active key changes. If no key is pressed, silence the piezo and restore the idle display.

6.2 Main functions

Function	Purpose
`setup()`	Initializes digital pin modes, serial communication, LCD dimensions, and three custom characters.
`showIdleScreen()`	Displays the default two-line prompt.
`showManualNote()`	Shows note name, solfège, and frequency for a manual key.
`danceGirl()`	Selects one of three custom LCD character frames using the note index modulo three.
`playNote()`	Displays a dance frame, calls `tone()`, waits for the specified duration, then calls `noTone()`.
`playSong()`	Loops through a song's melody and timing arrays, then restores the idle screen.
`playABC()`	Selects the ABC melody and duration arrays and sends them to `playSong()`.
`playMary()`	Selects the Mary melody and duration arrays.

Function	Purpose
`manualPiano()`	Checks all eight notes from C4 through C5, plays the selected frequency, and updates the LCD on changes.
`loop()`	Detects song-button presses, invokes the correct mode, and repeatedly scans manual inputs.

6.3 Readable code and software design decisions

Descriptive constants such as `NOTE_C4` and `c5Note` make pin and frequency mappings easier to audit. Melody data are stored separately from playback logic, avoiding a long sequence of repeated tone commands. `lastManualNote` prevents unnecessary LCD clearing while a key remains pressed. Song buttons use previous-state checks plus a short debounce delay to reduce repeated triggers. During a programmed song, the use of `delay()` makes playback blocking; the manual keyboard cannot be played simultaneously until the song finishes. Also, the code plays only one manual pitch at a time because the note-selection logic is an `if/else if` chain.

7 Results, Verification, and Rubric Alignment

Requirement	Evidence to present
C-major scale C4–C5	Eight manual buttons mapped to frequencies 262–523 Hz; demonstrate each in the video.
Code readability	Named constants, section comments, and functions in Appendix A.
Aesthetic feature	16×2 LCD note display and dancing-character animation.
Technical documentation	Component list, theory, pseudocode, actual schematic, and construction photos.
Video demonstration	Submit the separately recorded video showing the piano and both songs.

The code and circuit design support the required octave, but photographs, screenshots, and a video are separate evidence. The report does not substitute for testing each physical key and verifying both song buttons on the completed hardware.

8 Reflection on ChatGPT-Assisted Development

ChatGPT was useful as a design and coding assistant for organizing pin assignments, explaining the C-major scale, drafting Arduino functions, and suggesting LCD display features. It also helped identify the difference between `INPUT` with external pull-downs and `INPUT_PULLUP`, which was central to resolving the reversed C5 button behavior. Generating an initial code structure and technical-report outline reduced time spent on repetitive formatting.

However, AI-generated code cannot verify breadboard wiring or confirm that a physical switch is connected correctly. Earlier assumptions about C5 wiring caused the software to interpret the button

in reverse. The debugging process required observing the actual behavior, testing the digital input, and matching the final sketch to the circuit. The ABC song also demonstrates a limitation: sharing a tune with "Twinkle Twinkle Little Star" does not automatically guarantee lyric-accurate alphabet phrasing. AI suggestions should therefore be treated as drafts to inspect, compile, test, and revise, not as proof of successful hardware performance.

9 Conclusion

The final design combines eight manual notes spanning C4 to C5, two automatic melody selections, audible piezo output, and LCD feedback. The project demonstrates digital input design, pull-down versus pull-up logic, frequency-based sound generation, basic software modularity, and the importance of hardware/software integration. Completing the submission requires adding authentic photographs and screenshots and attaching the demonstration video.

A Complete Arduino Sketch

The following listing is included directly from the Arduino source supplied for this report. The two melody arrays and all eight manual-key conditions are retained. The complete Arduino source code is embedded below, so this report compiles without a separate code file. The accompanying `Arduino_Piano_Code.ino` is also provided for uploading to the Arduino IDE.

```
#include <LiquidCrystal.h>

// ============================================
// ARDUINO PIANO - ENES100
// 8 Manual Keys (C4 to C5)
// 2 Automatic Songs
// 16x2 LCD with Dancing Girl
// ============================================

// Manual piano button pins
const int cNote = 9;
const int dNote = 8;
const int eNote = 7;
const int fNote = 6;
const int gNote = 5;
const int aNote = 4;
const int bNote = 3;
const int c5Note = 12;

// Piezo buzzer
const int Piezo = 2;

// Automatic song buttons
const int abcButton = 10;
const int maryButton = 11;

// LCD pins: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(A0, A1, A2, A3, A4, A5);

// ============================================
// MUSICAL NOTE FREQUENCIES
// ============================================

const int NOTE_C4 = 262;
const int NOTE_D4 = 294;
const int NOTE_E4 = 330;
const int NOTE_F4 = 349;
```

```
const int NOTE_G4 = 392;
const int NOTE_A4 = 440;
const int NOTE_B4 = 494;
const int NOTE_C5 = 523;

// ============================================
// DANCING GIRL CHARACTERS
// ============================================

byte girl1[8] = {
  B01110,
  B01110,
  B00100,
  B11111,
  B10101,
  B01110,
  B01010,
  B10001
};

byte girl2[8] = {
  B01110,
  B01110,
  B00100,
  B11100,
  B10110,
  B01110,
  B10010,
  B00101
};

byte girl3[8] = {
  B01110,
  B01110,
  B00100,
  B00111,
  B01101,
  B01110,
  B01001,
  B10100
};

// ============================================
// ABC ALPHABET SONG
// ============================================
// Traditional ABC song melody

const int abcMelody[] = {

  NOTE_C4, NOTE_C4, NOTE_G4, NOTE_G4,
  NOTE_A4, NOTE_A4, NOTE_G4,

  NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4,
  NOTE_D4, NOTE_D4, NOTE_C4,

  NOTE_G4, NOTE_G4, NOTE_F4, NOTE_F4,
  NOTE_E4, NOTE_E4, NOTE_D4,

  NOTE_G4, NOTE_G4, NOTE_F4, NOTE_F4,
  NOTE_E4, NOTE_E4, NOTE_D4,

  NOTE_C4, NOTE_C4, NOTE_G4, NOTE_G4,
  NOTE_A4, NOTE_A4, NOTE_G4,

  NOTE_F4, NOTE_F4, NOTE_E4, NOTE_E4,
  NOTE_D4, NOTE_D4, NOTE_C4
};

const int abcDuration[] = {

```

```
  300,300,300,300,300,300,600,
  300,300,300,300,300,300,600,

  300,300,300,300,300,300,600,
  300,300,300,300,300,300,600,

  300,300,300,300,300,300,600,
  300,300,300,300,300,300,600
};

// ============================================
// MARY HAD A LITTLE LAMB
// ============================================

const int maryMelody[] = {

  NOTE_E4, NOTE_D4, NOTE_C4, NOTE_D4,
  NOTE_E4, NOTE_E4, NOTE_E4,

  NOTE_D4, NOTE_D4, NOTE_D4,
  NOTE_E4, NOTE_G4, NOTE_G4,

  NOTE_E4, NOTE_D4, NOTE_C4, NOTE_D4,
  NOTE_E4, NOTE_E4, NOTE_E4,

  NOTE_E4, NOTE_D4, NOTE_D4,
  NOTE_E4, NOTE_D4, NOTE_C4
};

const int maryDuration[] = {

  300,300,300,300,300,300,600,
  300,300,600,300,300,600,

  300,300,300,300,300,300,600,
  300,300,300,300,300,600
};

// ============================================
// BUTTON STATES
// ============================================

bool lastABCState = HIGH;
bool lastMaryState = HIGH;

// -1 means no manual note is pressed
int lastManualNote = -1;

// ============================================
// SETUP
// ============================================

void setup() {

  // All 8 manual keys use external
  // 10k ohm pull-down resistors.
  // Pressed = HIGH
  // Released = LOW

  pinMode(cNote, INPUT);
  pinMode(dNote, INPUT);
  pinMode(eNote, INPUT);
  pinMode(fNote, INPUT);
  pinMode(gNote, INPUT);
  pinMode(aNote, INPUT);
  pinMode(bNote, INPUT);

  // FIXED: C5 now behaves like other keys
  pinMode(c5Note, INPUT);

```

```
  // Song buttons connect to GND
  // Pressed = LOW
  pinMode(abcButton, INPUT_PULLUP);
  pinMode(maryButton, INPUT_PULLUP);

  pinMode(Piezo, OUTPUT);

  Serial.begin(9600);

  // Initialize LCD
  lcd.begin(16, 2);

  // Create dancing girl characters
  lcd.createChar(0, girl1);
  lcd.createChar(1, girl2);
  lcd.createChar(2, girl3);

  showIdleScreen();
}

// ============================================
// DEFAULT LCD SCREEN
// ============================================

void showIdleScreen() {

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("ARDUINO PIANO");

  lcd.setCursor(0, 1);
  lcd.print("PLAY A NOTE!");
}

// ============================================
// DISPLAY PRESSED NOTE
// ============================================

void showManualNote(const char* noteName,
                    const char* solfege,
                    int frequency) {

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("PLAYING: ");
  lcd.print(noteName);

  lcd.setCursor(0, 1);
  lcd.print(solfege);
  lcd.print(" ");
  lcd.print(frequency);
  lcd.print(" Hz");

  Serial.print("Playing: ");
  Serial.println(noteName);
}

// ============================================
// DANCING GIRL ANIMATION
// ============================================

void danceGirl(int frame) {

  lcd.setCursor(7, 1);
  lcd.write((uint8_t)(frame % 3));
}

// ============================================
```

```
// PLAY ONE SONG NOTE
// ============================================

void playNote(int frequency,
              int duration,
              int frame) {

  danceGirl(frame);

  tone(Piezo, frequency);

  delay(duration);

  noTone(Piezo);

  delay(80);
}

// ============================================
// PLAY AUTOMATIC SONG
// ============================================

void playSong(const int melody[],
              const int durations[],
              int length,
              const char* songName) {

  noTone(Piezo);

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print(songName);

  lcd.setCursor(0, 1);
  lcd.print("DANCING");

  for (int i = 0; i < length; i++) {

    playNote(
      melody[i],
      durations[i],
      i
    );
  }

  noTone(Piezo);

  lastManualNote = -1;

  showIdleScreen();
}

// ============================================
// PLAY ABC SONG
// ============================================

void playABC() {

  Serial.println("Playing ABC Song");

  int length =
    sizeof(abcMelody) / sizeof(abcMelody[0]);

  playSong(
    abcMelody,
    abcDuration,
    length,
    "ABC SONG"
  );
```

```
}

// ==========================================
// PLAY MARY HAD A LITTLE LAMB
// ==========================================

void playMary() {

  Serial.println("Playing Mary Had a Little Lamb");

  int length =
    sizeof(maryMelody) / sizeof(maryMelody[0]);

  playSong(
    maryMelody,
    maryDuration,
    length,
    "MARY HAD A LAMB"
  );
}

// ==========================================
// MANUAL PIANO: C4 THROUGH C5
// ==========================================

void manualPiano() {

  int currentNote = -1;
  int frequency = 0;

  const char* noteName = "";
  const char* solfege = "";

  // C4 - Button 1
  if (digitalRead(cNote) == HIGH) {

    currentNote = 0;
    frequency = NOTE_C4;
    noteName = "C4";
    solfege = "DO";
  }

  // D4 - Button 2
  else if (digitalRead(dNote) == HIGH) {

    currentNote = 1;
    frequency = NOTE_D4;
    noteName = "D4";
    solfege = "RE";
  }

  // E4 - Button 3
  else if (digitalRead(eNote) == HIGH) {

    currentNote = 2;
    frequency = NOTE_E4;
    noteName = "E4";
    solfege = "MI";
  }

  // F4 - Button 4
  else if (digitalRead(fNote) == HIGH) {

    currentNote = 3;
    frequency = NOTE_F4;
    noteName = "F4";
    solfege = "FA";
  }

  // G4 - Button 5
```

```
388   else if (digitalRead(gNote) == HIGH) {
389
390     currentNote = 4;
391     frequency = NOTE_G4;
392     noteName = "G4";
393     solfege = "SOL";
394   }
395
396   // A4 - Button 6
397   else if (digitalRead(aNote) == HIGH) {
398
399     currentNote = 5;
400     frequency = NOTE_A4;
401     noteName = "A4";
402     solfege = "LA";
403   }
404
405   // B4 - Button 7
406   else if (digitalRead(bNote) == HIGH) {
407
408     currentNote = 6;
409     frequency = NOTE_B4;
410     noteName = "B4";
411     solfege = "TI";
412   }
413
414   // C5 - Button 8
415   // FIXED: Pressed = HIGH
416   else if (digitalRead(c5Note) == HIGH) {
417
418     currentNote = 7;
419     frequency = NOTE_C5;
420     noteName = "C5";
421     solfege = "DO";
422   }
423
424   // ========================================
425   // PLAY PRESSED NOTE
426   // ========================================
427
428   if (currentNote != -1) {
429
430     // Produce sound
431     tone(Piezo, frequency);
432
433     // Update LCD only if note changed
434     if (currentNote != lastManualNote) {
435
436       showManualNote(
437         noteName,
438         solfege,
439         frequency
440       );
441
442       lastManualNote = currentNote;
443     }
444   }
445
446   // ========================================
447   // NO BUTTON PRESSED
448   // ========================================
449
450   else {
451
452     noTone(Piezo);
453
454     if (lastManualNote != -1) {
455
456       showIdleScreen();
457
```

```
      lastManualNote = -1;
    }
  }
}

// ==========================================
// MAIN LOOP
// ==========================================

void loop() {

  bool abcState = digitalRead(abcButton);
  bool maryState = digitalRead(maryButton);

  // ========================================
  // D10: ABC ALPHABET SONG
  // ========================================

  if (abcState == LOW &&
      lastABCState == HIGH) {

    delay(30);

    if (digitalRead(abcButton) == LOW) {

      playABC();
    }
  }

  // ========================================
  // D11: MARY HAD A LITTLE LAMB
  // ========================================

  else if (maryState == LOW &&
           lastMaryState == HIGH) {

    delay(30);

    if (digitalRead(maryButton) == LOW) {

      playMary();
    }
  }

  // Update song button states
  lastABCState = digitalRead(abcButton);
  lastMaryState = digitalRead(maryButton);

  // ========================================
  // MANUAL PIANO MODE
  // ========================================

  if (lastABCState == HIGH &&
      lastMaryState == HIGH) {

    manualPiano();
  }
  else {

    noTone(Piezo);
  }

  delay(10);
}
```

B Code Walkthrough by Section

1. **Library and pins:** `LiquidCrystal.h` provides the display interface. The initial constants map the eight manual buttons, piezo, and two song buttons to physical pins.
2. **Note constants:** Eight integer frequencies specify C4 through C5, including the required 523-Hz high C.
3. **Custom LCD bytes:** The `girl1`, `girl2`, and `girl3` arrays each define an eight-row, five-pixel-wide LCD character.
4. **Melody and duration arrays:** `abcMelody`/`abcDuration` and `maryMelody`/`maryDuration` hold pitches and millisecond durations in matching positions.
5. **State variables:** `lastABCState` and `lastMaryState` track song-button transitions; `lastManualNote` tracks the current key for efficient LCD updates.
6. **Initialization:** `setup()` uses `INPUT` for all eight pull-down-equipped keys, `INPUT_PULLUP` for the two song switches, and `OUTPUT` for the piezo.
7. **LCD routines:** `showIdleScreen()` writes the idle prompt, while `showManualNote()` prints the current pitch and its approximate frequency.
8. **Animation and sound:** `danceGirl()` chooses a frame; `playNote()` starts a pitch, waits, stops the sound, and inserts a short separation between notes.
9. **Automatic playback:** `playSong()` iterates over arrays; `playABC()` and `playMary()` pass the corresponding arrays and titles.
10. **Manual playback:** `manualPiano()` checks C4, D4, E4, F4, G4, A4, B4, then C5 on D12. Pressed keys use `HIGH`; no pressed key calls `noTone()`.
11. **Continuous operation:** `loop()` reads the song inputs, checks for a newly pressed button, and otherwise calls `manualPiano()`. The final short delay slows the polling loop slightly.

C TinkerCad Project Code and Physical Lab Demonstration

Arduino Source Code:

Click Here to View Arduino Code

TinkerCad Project link:

Click Here to View and play the project

Tinkercad and Real-Life Circuit Demonstration Video:

Click Here to Watch the Demonstration

The demonstration includes the circuit implementation, manual piano keys from C4 to C5, and both automatic songs. The LCD functionality is included in the Arduino code but was not used in the physical lab demonstration due to hardware issues.